

BROWN

**Deterministic Guardrails for Preserving Privacy and
Security in Real-World Computer Systems**

by
Justus Adam

B.Sc., Technische Universität Dresden, 2016
Sc.M., Technische Universität Dresden, 2020

A dissertation submitted in partial fulfillment of the
requirements for the Degree of Doctor of Philosophy
in the Department of Computer Science at Brown University

Providence, Rhode Island
May 2028

© Copyright 2028 by Justus Adam

This dissertation by Justus Adam is accepted in its present form
by the Department of Computer Science as satisfying the
dissertation requirement for the degree of Doctor of Philosophy.

Date _____
Malte Schwarzkopf, Advisor

Recommended to the Graduate Council

Date _____
Deepti Raghavan, Reader

Date _____
Akshay Narayan, Reader

Date _____
Shriram Krishnamurthi, Reader

Approved by the Graduate Council

Date _____
Sorin Istrail, Dean of the Graduate School

PREFACE AND ACKNOWLEDGMENTS

Thank people here.

THESIS STATEMENT

Shared, high-level vocabularies make enforcing privacy and security policies in applications tractable, especially at real-world scale and complexity.

There are many techniques for ensuring that programs are compliant with privacy and security requirement. However, while the concerns are paramount, reliable techniques are used only in ultra-high-danger domains. This is because such techniques are either inefficient or highly unwieldy to use. In this thesis I explore an approach that uses high-level vocabularies that makes sophisticated safeguards accessible to users and developers by meeting them where they are. At the same time, this vocabulary maps efficiently to sound enforcement techniques.

CONTENTS

1	Introduction	1
1.1	Trust and Threat Model	4
1.2	Contributions	5
2	Background	7
2.1	Data and Control Flow Analysis	7
2.2	Agentic AI	9
3	Paralegal	11
4	Palsgraf	19
5	Related Work and Conclusion	27
	Bibliography	28

LIST OF TABLES

Table 4.1	Which scenario variants each protection mechanism allowed or blocked. ✓ marks the intended outcome, ✗ a failure. The global sandbox is the honest baseline — a policy a careful engineer would write without per-task knowledge — and it fails in both directions, blocking intended work in one scenario and admitting divergent effects in the others. Only the per-task policy separates them everywhere.	25
-----------	---	----

LIST OF ILLUSTRATIONS

Figure 1.1	The two systems in this thesis instantiate one structure. In both, a vocabulary (middle row, highlighted) sits between the people who hold the relevant expertise and a mechanism that checks their intent without sharing it: the enforcer never interprets a marker or an action name, and no author needs to reason about PDGs or mount layouts. The systems differ in when the vocabulary is bound to the program — at build time for Paralegal, per task at run time for Palsgraf — and in whether the party supplying the code is trusted.	3
Figure 2.1	An agentic AI application uses an interpreter to call tools requested by an LLM	9
Figure 3.1	Paralegal’s workflow. A privacy engineer writes a policy over marker names; a developer attaches those markers to source entities. The analyzer builds a marked program dependence graph over the application and its libraries, and checks the policy against it. Neither author reasons about the graph, and the analyzer assigns no meaning to the names.	11
Figure 3.2	User-data deletion written in Paralegal’s policy language. Figure 3.2a is too coarse: it is satisfied by code that only builds a deletion query. Figure 3.2b additionally requires the query to reach an executes marker, which needs a new marker from the developer. This give-and-take between privacy engineer and developer is the common workflow, and it is why the vocabulary has to be cheap to extend.	12
Figure 3.3	Partial and simplified marked PDG for the deletion example. Solid rectangles are PDG nodes, dashed rectangles are function scopes, and the rounded boxes are markers. The red subgraph is the comment-deletion code that was missing. “ <i>Lk</i> ” refers to line <i>k</i> of Listing 3.1 ; “start” and “end” are the function entry and exit locations.	13
Figure 3.4	In the “Workspace Only” configuration, which analyzes only the crates in the current workspace, Paralegal’s end-to-end runtime is under 2.1 seconds for most applications — fast enough to run interactively. The exceptions are	

	Lemmy, which has 72 analysis entry points, and Hyperswitch, which is a large crate. PDG construction dominates.	17
Figure 3.5	In the “All Dependencies” configuration, which analyzes the application together with its dependencies, runtimes remain compatible with a CI budget.	17
Figure 3.6	Paralegal takes a mean of 0.8 seconds per endpoint, so cost scales with the number of analysis entry points rather than with total application size. ...	18
Figure 3.7	Adaptive approximation reduces end-to-end runtime: without it, PDG construction inlines call sites that cannot affect the policy’s verdict. This is what makes the interactive numbers above achievable.	18
Figure 4.1	An MCP tool for file creation. In Figure 4.1a the implementation performs an undisclosed metrics upload in addition to the documented file write; the description is the only thing the model — or the user approving its use — gets to see. Figure 4.1b adds a fine-grained sandbox whose configuration is injected by the trusted client rather than supplied by the server. Note that this is the design in which the tool developer cooperates; this chapter argues for confining the process instead, precisely because cooperation cannot be assumed.	19
Figure 4.2	Palsgraf’s components and trust boundaries, which are easy to invert. The client is trusted and is the only party that authors policy; the user approves it in the vocabulary of the action catalog. Everything the model asks for is mediated by the interpreter, and anything that escapes the interpreter — a spawned process, most of all — is confined by a sandbox the client configured, not by cooperation from the code being run.	20

CHAPTER 1

INTRODUCTION

Today's applications are ever-growing in size and complexity, and ever more user data is being processed through these systems. To ensure that the users to whom this data belongs are afforded privacy and that the computer systems are secure, engineers and architects conduct mainly laborious manual audits. Some code-analysis tools are in use also, but they lack the ergonomics and scalability to be applied broadly. With the advent of agentic AI, these problems are exacerbated as developers increasingly do not write the code themselves. Code reviews become even more laborious, as LLMs generate vast amounts of code in minimal time. In many cases, the core of the applications themselves becomes agentic AI, a fundamentally unpredictable component.

Human reasoning alone will not efficiently scale to the complexity of modern applications. At the same time, automated tools must deliver reliable results, which disqualifies approaches that use AI on the critical path. This leaves formal techniques, such as static analysis, proofs, sandboxing or instrumentation. Traditionally, these are used only in high-impact areas as they require additional expertise to use effectively. They often struggle to scale to large code bases and they are ill-adapted to the dynamic nature of AI enabled applications.

This thesis comprises two projects that address roadblocks in the application of formal guardrails to realistic applications. Paralegal is a static analyzer for Rust programs. It presents an ergonomic interface to privacy and security policy authoring based on

markers, text objects attached to source code elements. For classical applications, static analysis offers the benefit of providing guarantees at compile time, and do not impact runtime performance or alter runtime behavior in surprising ways. The enforcement engine, a program dependence graph (PDG) based analyzer, uses these markers as well as guarantees provided by Rust's type-system enforced ownership model to make the analysis scale to real-world scale Rust programs.

Palsgraf on the other hand addresses the novel threat, posed to users, from the addition of agentic AI. Applications that use agentic AI are highly dynamic. Their strength lies in performing tasks that are, at build time, unanticipated, and at creating tools for solving these tasks on-the-fly. This makes static techniques ill suited for ensuring an AI enabled application preserves user privacy and security without eliminating the adaptability for which the AI was employed in the first place.

Dynamic techniques also struggle in the agentic AI setting. Traditionally, the dynamism for these techniques is related to runtime inputs, whereas policies are statically fixed. With agentic AI, the tasks performed are not known ahead of time and only task-specific policies can distinguish desired behavior from dangerous violations.

Palsgraf is a dynamic policy enforcement engine that with first-class support for task-specific policies. To ensure the policies are trustworthy Palsgraf uses the user as source of the policies. Creating format task-specific policies takes however a lot of effort. To alleviate this burden Palsgraf prompts the agent itself to draft the policy. Palsgraf then renders this policy in human readable form to the user for approval. This mechanism is predicated on a policy that is concise and easy for users to understand. To facilitate this Palsgraf uses an expert-authored, registry of parameterized shapes of shell commands and network requests. Each such shape is given a meaningful name that the user would recognize. For example, the `GIT_COMMIT(where="/projects/foo")` is a meaningful effect to a typical developer and they need not reason about the fact that this means, internally, that this rule will forbid the `--force` flag.

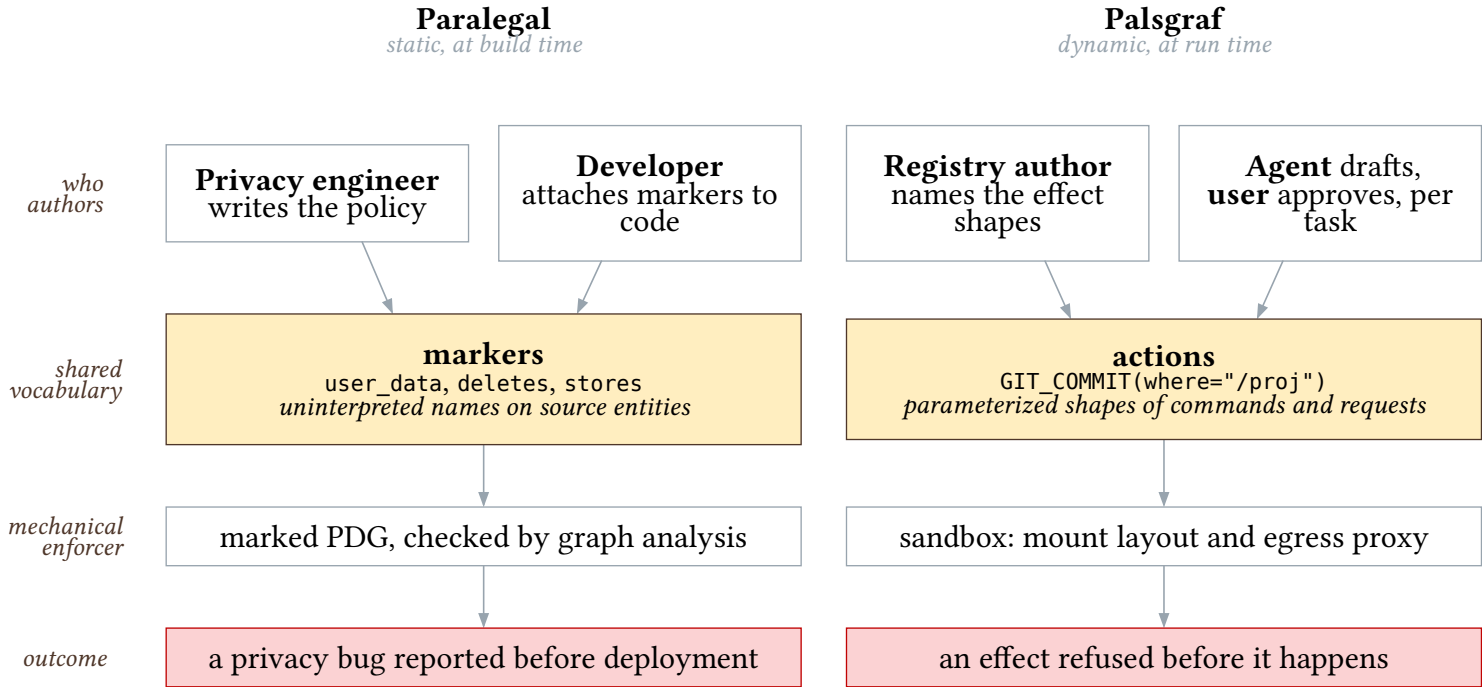


Figure 1.1. The two systems in this thesis instantiate one structure. In both, a vocabulary (middle row, highlighted) sits between the people who hold the relevant expertise and a mechanism that checks their intent without sharing it: the enforcer never interprets a marker or an action name, and no author needs to reason about PDGs or mount layouts. The systems differ in when the vocabulary is bound to the program — at build time for Paralegal, per task at run time for Palsgraf — and in whether the party supplying the code is trusted.

TODO: 2. The common thread, made explicit + a running example. The intro now describes two projects back to back but never says what makes them one thesis. Add a section (or a closing paragraph) that:

- Puts markers and actions side by side as the *same* construct: a name that a non-expert already understands, attached to a program entity, that an enforcer can check mechanically. `user_data/deletes` and `GIT_COMMIT(where=...)` are the same idea at two different binding times.
- Picks one or two concrete incidents to carry through the whole thesis. Best candidates: Plume’s missing comment deletion (the Paralegal running example, already has code, policy and PDG figures) and the Railway volume-delete incident (`investigations/11-volume-delete-incident/`) for Palsgraf. Both are real, both are one-sentence explainable, and both fail for the same reason: the person who could have caught it was not shown the effect in terms they reason about.
- States the shape of the answer once, then says each chapter instantiates it.

Figure: Figure 1.1 is drawn and placed above — read row-wise, the rows are this argument. The prose should walk it once rather than restate it, and the sentence that matters is the one about the enforcer never interpreting a name.

1.1 Trust and Threat Model

The assumption for this work is that users can be trusted to specify policies, but they may fail to produce code that upholds them. We do not trust agents to produce correct policies or correct implementations, though we need their cooperation to make progress.

For Paralegal, we trust a knowledgeable policy engineer to specify privacy and security policies, define markers and document what those markers mean. Developers, as experts in the application, are trusted to apply the markers to code entities. Paralegal’s goal is to help developers that make mistakes when writing the actual application. As

such, we do not trust the developer to get the implementation right, especially in an evolving code base. The developers are trusted to place the markers correctly, since that is a *rare* and *high-impact* action and thus warrants careful treatment and review.

Palsgraf *guarantees* that every effect a task’s execution produces is bound by the policy in effect at that time. The system does not ensure that the executed effects achieve the task or that the LLM’s outputs are correct. Side-channels, kernel compromise and sandbox failure are out of scope.

The TCB encompasses the sandbox, the kernel that supports the sandbox as well as the network and shell brokers. The system does not trust the user’s (possibly copy-pasted) prompt, but it trusts the user to review proposed policies and it trusts the on-disk actions catalogue’s web request predicates, shell command footprint specifications and policy files.

The *agent is untrusted* and Palsgraf relies on it only for utility. The agent can get confused about what goal it is supposed to achieve or attempt unsafe means to achieve it. An *adversary* may hijack the agent by altering web responses from compromised web servers or by placing content in files the agent legitimately needs to read, such as a public code repository. The adversary may modify the user’s machine only via the agent. Both adversary and agent have full knowledge of the policy.

1.2 Contributions

This work make the following contributions:

1. The Paralegal static analyzer, which checks high-level properties against low-level, evolving code bases.
2. The marker abstraction to decouple policies and code; and techniques to efficiently generate precise semantic PDGs from Rust code and model the behavior of library code.
3. A flexible policy framework that compiles policies in a high-level language into queries on semantic PDGs.

4. Case studies reporting on our experience of applying Paralegal to eight real-world Rust web applications.
5. A policy language and action verb mechanism that lets users express policies at the level of meaningful effect units.
6. An scheme and mechanism for agents to draft policies and users to review them, compatible with MCP.
7. A sandbox adaptation that allows judging network requests on payload and protocol program arguments.

TODO: 4. Contributions and roadmap. Standard, but missing, and it is where the two-projects-one-thesis claim gets made concrete. One numbered list of contributions (marker abstraction + PDG-based checking + the real bugs found; action registry + sandbox enforcement + agent-drafts/user-approves loop; and the cross-cutting claim about vocabularies), then a one-paragraph chapter map. Note which chapters derive from published work (the OSDI'25 Paralegal paper) and state the authorship split for the co-authored material, which the Graduate School expects.

CHAPTER 2

BACKGROUND

2.1 Data and Control Flow Analysis

Dataflow analysis is crucial when determining the privacy compliance of programs. Whether a datum is private cannot always be determined by its content. PII (name, SSN, telephone number) can sometimes be identified via its unique structure. However, private data is a broader category, and data such as diary or email content, bank balances or contracted illnesses are often embedded in unstructured prose which also colocates data that may be sufficient to determine the data subject.

A reliable way to determine if data is private is by considering the data source, e.g. a bank statement, patient chart etc. This means however that in a program that manipulates such data, it cannot be determined at the sink whether the data that was input to the operation is private **unless** its origins have been tracked through the program.

A primary challenge in statically determining the origins of a datum in a program are pointers. Which value a given pointer may refer to, and therefore whether one of them could be sensitive or not, depends on all parts of the program that may modify pointer. This is called alias-analysis and unlike many other parts of dataflow analysis it is a non-local analysis, meaning it cannot be performed for one function alone, but must take into account all of its callers. This leads to an explosion of the state space.

TODO: 5. Finish the aliasing story — Rust as the escape hatch. This section stops at “the state space explodes” and never says how Paralegal gets out. Continue with:

- Ownership and borrowing as a *locally checkable* aliasing discipline: a `&mut` is unique, so a callee’s writes are bounded by its signature. This is the thing that lets a modular analysis stay sound without a whole-program alias analysis.
- MIR as the analysis target — post-monomorphization, post-desugaring, why that matters for precision and for handling generics/trait dispatch.
- Where the discipline breaks: `unsafe`, interior mutability, FFI, raw pointers. Be honest about what Paralegal assumes here; it is the first thing a reader will poke at.
- Short contrast with the Java/Android lineage (FlowDroid, DroidSafe) to make the point that the language choice is doing analytical work, not just engineering convenience.

TODO: 6. New section: Policy languages and information-flow control. This is where you establish that “shared vocabulary” is the axis nobody else optimized for. Write about:

- The IFC lineage: label lattices, Jif, HLIO; then the dynamic/application line — Resin, Jacqueline, Riverbed, Sesame; then the compliance-oriented line — PrivGuard, Rule-Keeper.
- The recurring shape: an expressive labeling discipline that requires the policy author to also be a programming-languages expert, or that is expressive only over noninterference-shaped properties.
- The thesis’s wedge: many real privacy properties are *not* noninterference (deletion completeness, expiration, authorization, third-party sharing). A marker vocabulary plus reachability over a PDG expresses them; a lattice does not. This is the argument the Paralegal chapter’s comparison table (against IFC and CodeQL) cashes out, so set it up here.

TODO: 7. New section: Confinement and sandboxing. Background for the Palsgraf chapter, and the second half of the vocabulary argument. Cover:

- The mechanism layer: OS namespaces and overlays, bubblewrap, Seatbelt/SBPL, seccomp; and the in-process line for contrast — NaCl, RLBox, WebAssembly, Ryoan.
- The observation that reframes them: these mechanisms are configured in *kernel nouns* — paths, mounts, syscalls, hosts — while the person granting permission thinks in *task nouns*. A user can meaningfully approve “commit to this repo” and cannot meaningfully approve a mount layout. That gap is exactly the gap markers close on the static side.
- Why coarse sandboxes are the honest baseline and where they fail: a project-wide allow list either blocks benign work or admits the harmful case; your coarse-sandbox protection column is empirical evidence for this, so foreshadow it.

2.2 Agentic AI

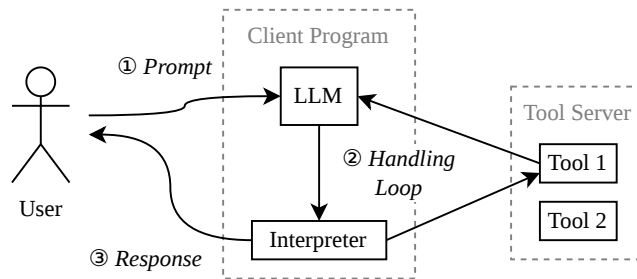


Figure 2.1. An agentic AI application uses an interpreter to call tools requested by an LLM

The basic architecture or agentic loops is remarkably simple and, crucially, has deterministic programs on the critical path. Conceptually, agentic AI lets a language model “invoke tools”, that is, programs. However, in reality, the AI itself has no means of directly running these programs. Its inputs and outputs are simply text [1, 2]. A special interpreter processes the text produced by the LLM and *this interpreter* invokes programs in response to LLM requests. A common architecture is that the LLM returns all of its responses

in JSON format, with a `type` field that identifies the request as, for example, `thinking`, `user_response` or `tool_call`. Depending on this choice the interpreter performs dispatch back to the LLM (`thinking`), to the UI renderer (`user_response`) or a program (`tool_call`). This means the interpreter retains full control over how the tool call proceeds. It can, for example, log the request, inspect or change its arguments, fabricate the response or outright deny it.

TODO: 8. Extend this section: the agentic threat landscape, and why model-level defenses are not enforcement. You have established that a deterministic interpreter sits on the critical path — good, that is the opening. Now write the threat side:

- The attack classes, with the citations already in `refs.yaml`: prompt injection (OWASP LLM01), sensitive-information disclosure (LLM02), adversarial SEO, universal/trans-ferable jailbreaks. Then the supply-chain surface that is specific to agents: MCP servers, skills, ClawHub, and the OpenClaw CVEs and issues you have collected.
- The accident class, which is distinct and probably more common: the Railway volume delete, the Replit production-database wipe, the SABER operational- safety results. Make the point that guardrails must catch both, and that the accident case is what makes user-facing legibility (not just isolation) the requirement.
- Why alignment, system prompts and in-agent rules are advisory: cite the statistical-jailbreaking result and the volume-delete confession, where the agent quotes the rule it violated. This is the paragraph that justifies “deterministic” in the title.
- Position the existing defenses so the reader knows what is new: CaMeL, Fides, ACE, Progent, IsolateGPT, Consecra, and the permission-prompt status quo (Claude Code, VS Code). Sort them by *who authors the policy* and *in what vocabulary* — that framing is your contribution and it makes the related-work chapter mostly write itself.

Figure: the existing `assets/0overview.pdf` covers the loop; consider a second one overlaying where each defense intercepts (model, interpreter, syscall).

CHAPTER 3

PARALEGAL

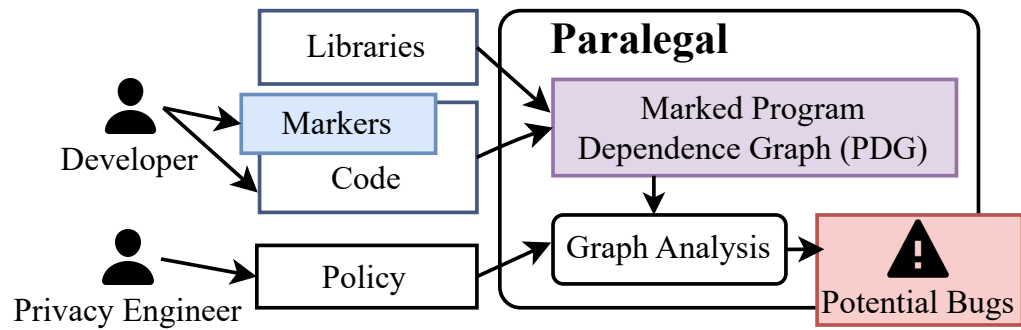


Figure 3.1. Paralegal’s workflow. A privacy engineer writes a policy over marker names; a developer attaches those markers to source entities. The analyzer builds a marked program dependence graph over the application and its libraries, and checks the policy against it. Neither author reasons about the graph, and the analyzer assigns no meaning to the names.

```

1  #[paralegal::marker(user_data)]
2  struct Comment { ... }
3
4  impl Database {
5      #[paralegal::marker(make_delete_query, arguments = [id])]
6      fn prepare_delete(&mut self, id: u32, table: &str) {...}
7  }
8
9  impl User {
10     #[paralegal::analyze]
11     fn delete_user(&self, db: &mut Database) {
12         let my_data: UserData = self.get_my_data();
13         db.prepare_delete(self.id, "users");
14         for post in &my_data.posts {
15             db.prepare_delete(post.id, "posts");
16         }
17         for comment in &my_data.comments {
18             db.prepare_delete(comment.id, "comments");
19         }
20         db.execute();
21     }
22 }

```

} the bug

Listing 3.1. Paralegal alerts developers to the missing code (red) that deletes a user’s comments when deleting their account in Plume. Markers are highlighted in teal. Code simplified and error handling omitted.

Somewhere:

1. For each "user data" type marked user_data:
 - A. There is a "source" that produces "user data" where:
 - a. There is a "deleter" marked deletes where:
 - i) "source" goes to "deleter"

(a) The initial policy.

Somewhere:

1. For each "user data" type marked user_data:
 - A. There is a "source" that produces "user data" where:
 - a. There is a "deleter" marked make_delete_query where:
 - i) "source" goes to "deleter"
 - and
 - ii) There is an "execute" marked executes where:
 - A) "deleter" goes to "execute"

(b) The revised policy.

Figure 3.2. User-data deletion written in Paralegal’s policy language. **Figure 3.2a** is too coarse: it is satisfied by code that only builds a deletion query. **Figure 3.2b** additionally requires the query to reach an executes marker, which needs a new marker from the developer. This give-and-take between privacy engineer and developer is the common workflow, and it is why the vocabulary has to be cheap to extend.

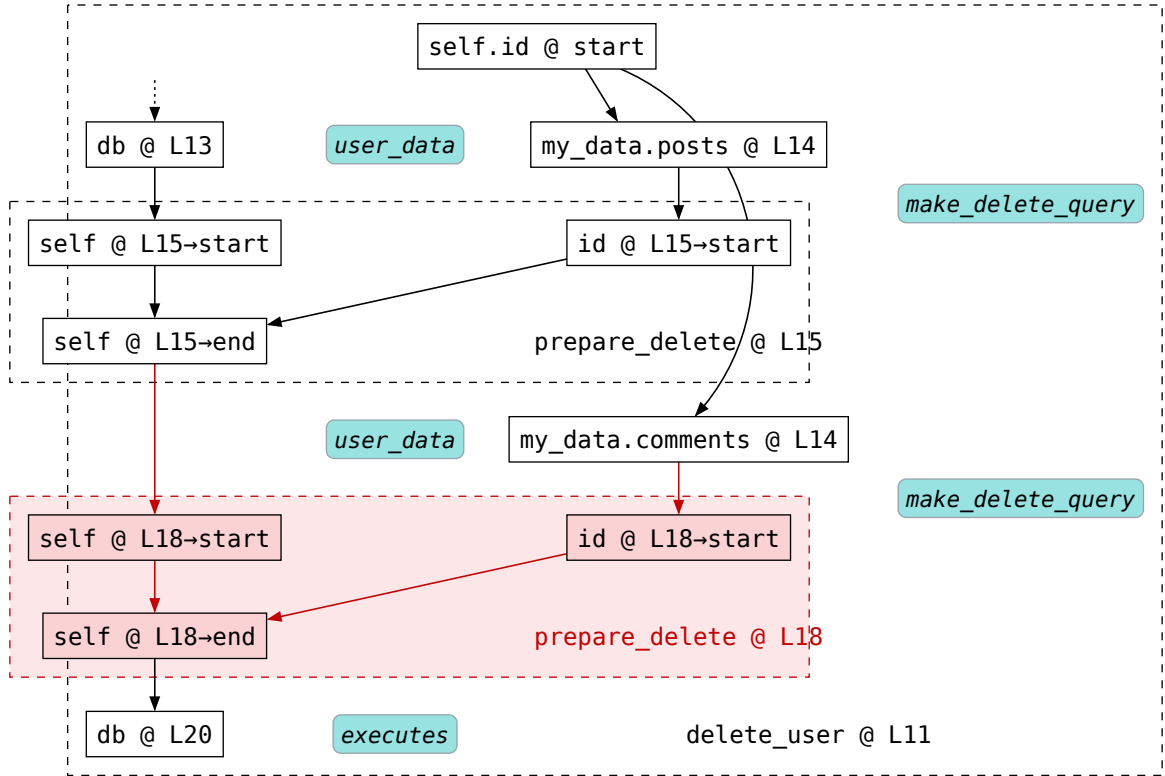


Figure 3.3. Partial and simplified marked PDG for the deletion example. Solid rectangles are PDG nodes, dashed rectangles are function scopes, and the rounded boxes are markers. The red subgraph is the comment-deletion code that was missing. “ Lk ” refers to line k of Listing 3.1; “start” and “end” are the function entry and exit locations.

TODO: 9. Markers as the vocabulary, and the division of labor. Lead the chapter with the social structure, not the algorithm — it is the part that generalizes to [Chapter 4](#) and the part the OSDI paper itself calls the key to practicality. Write about:

- The three roles and what each one must know: privacy engineers write policies over marker names and know the regulation; developers apply markers to source entities and know the code; the analyzer knows neither and needs no semantics for the names at all. Markers are deliberately *uninterpreted* — say that explicitly, it is what makes the split possible.
- What a marker can attach to (types, fields, arguments, returns, call sites) and why that granularity is the right unit.
- Marker economics: how many are needed per application, who writes which, what happens as code evolves. The ergonomics evaluation already answers this; the argument to make here is that the annotation burden is bounded and amortized, because that is the objection to every annotation-based system.
- The give-and-take loop from the Plume example: the first policy is wrong, the error message shows why, the policy and markers get revised together. This is the honest workflow and it is more convincing than a clean story.

Figures placed: [Figure 3.1](#) (the workflow, verbatim from the paper), [Listing 3.1](#) (the code, with the missing deletion highlighted and annotated), and [Figure 3.2](#) — the before/after policy pair, which carries the give-and-take point better than prose. Point the text at [Figure 3.2a](#) and [Figure 3.2b](#) individually. *Still to port* (a LaTeX table, so it needs retyping): `fig:apps` from `chapters/prototype.tex` — the applications table with LoC, policies, marked locations and entry points.

TODO: 10. The mechanism: marked PDGs and the policy language. The technical core. Cover, roughly in this order:

- PDG construction over MIR: what the nodes and edges are, data versus control dependence, how library code is modeled through ownership rather than analyzed, and where the analysis is modular versus whole-program.
- Adaptive approximation and inconsequential call-site removal — the two optimizations that make it scale. Do not bury these; they are the reason the numbers are interactive rather than overnight, and one of them (Contile) determines whether an application is analyzable at all.
- The policy language: the controlled-natural-language surface, its quantifier structure, what it compiles to, and the grammar. Argue the CNL choice on audience grounds (privacy engineers, not PL people), which is the same argument as the marker one.
- Error messages as a first-class design concern — the Plume deletion error is a good concrete artifact to show.

Figures: **Figure 3.3** is redrawn in Typst and placed above — it is the one picture that makes “marked PDG” concrete, so the walkthrough of PDG construction should be written against it. Note its function scope is labelled `⊔11`; the paper’s drawing said `⊔8`, which was stale. *Still to port:* the PDG/CFG grammar figure (`fig:grammar, chapters/pdg.tex`), which maps cleanly onto a Typst table.

TODO: 11. Evaluation, limits, and the bridge to Chapter 4. Write about:

- The bugs: seven across Plume, Atomic and Lemmy, two previously unknown. Lead with these; they are the result.
- Expressiveness against IFC and CodeQL — this is the section that cashes out Chapter 2's claim that real privacy properties are not noninterference.
- Performance, framed as a deployment claim rather than a benchmark: under interactive latency in the workspace configuration, tolerable in CI over all dependencies.
- Ergonomics and maintenance under code evolution.
- *Then the limits, stated by you rather than left to a reader:* markers are trusted, not verified; the analysis is static so it says nothing about values or runtime configuration; unsafe and FFI are assumptions; applications must be Rust. Each of these is a reason Palsgraf is a different system rather than an extension — end the chapter by making that argument, since it is the hinge of the whole thesis.

Figures: the four performance plots below are placed already. *Still to port* (LaTeX tables, so they need retyping): the bug table from `chapters/case-studies.tex`, the applications table `fig:apps` from `chapters/prototype.tex`, the IFC/CodeQL comparison `tab:results` from `chapters/eval-related-work.tex`, and the grammar `fig:grammar` from `chapters/pdg.tex`.

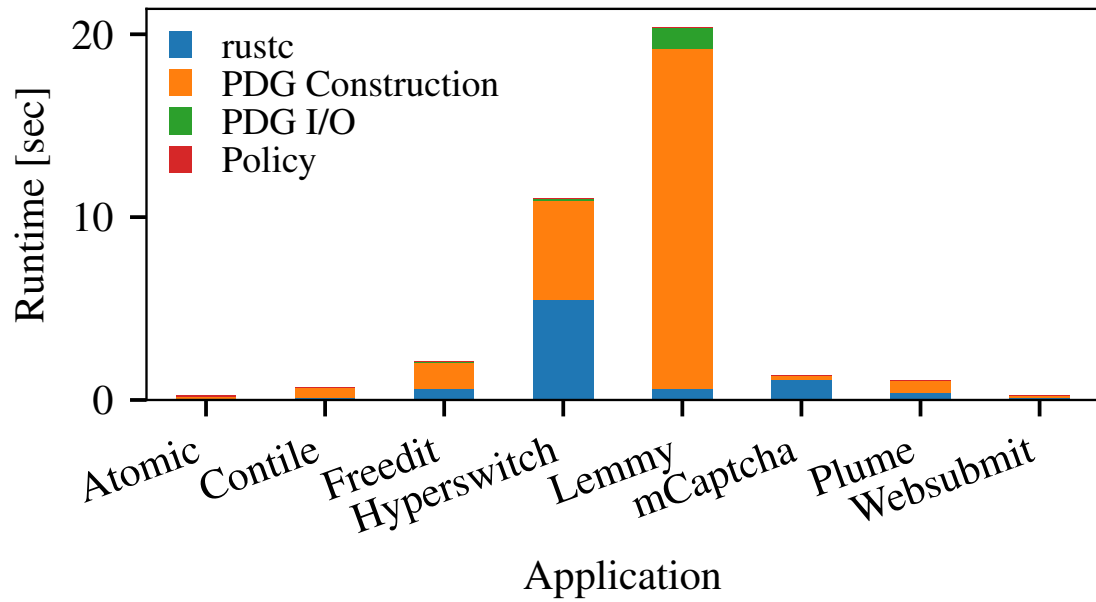


Figure 3.4. In the “Workspace Only” configuration, which analyzes only the crates in the current workspace, Paralegal’s end-to-end runtime is under 2.1 seconds for most applications — fast enough to run interactively. The exceptions are Lemmy, which has 72 analysis entry points, and Hyperswitch, which is a large crate. PDG construction dominates.

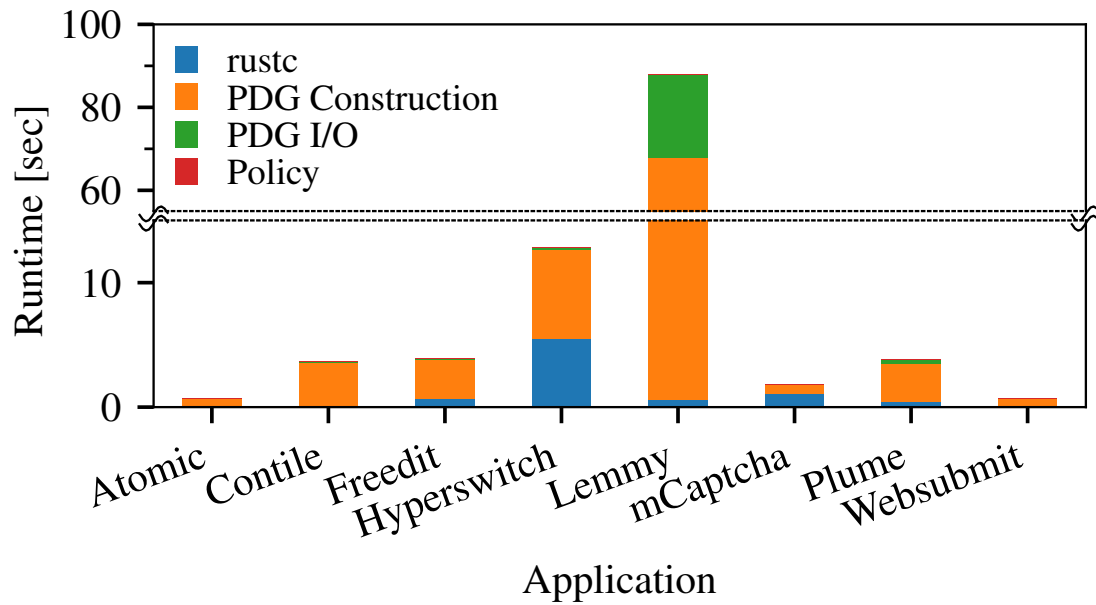


Figure 3.5. In the “All Dependencies” configuration, which analyzes the application together with its dependencies, runtimes remain compatible with a CI budget.

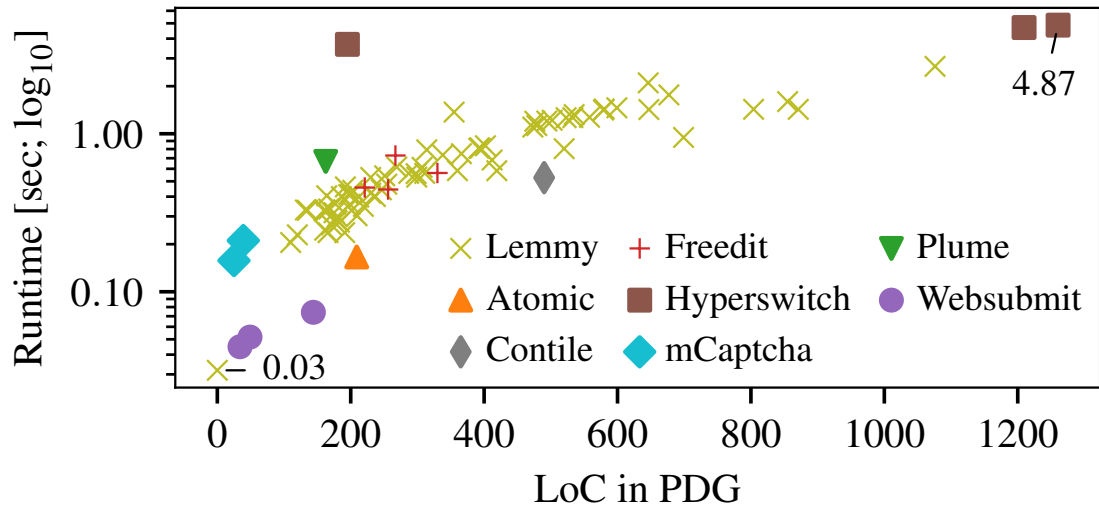


Figure 3.6. Paralegal takes a mean of 0.8 seconds per endpoint, so cost scales with the number of analysis entry points rather than with total application size.

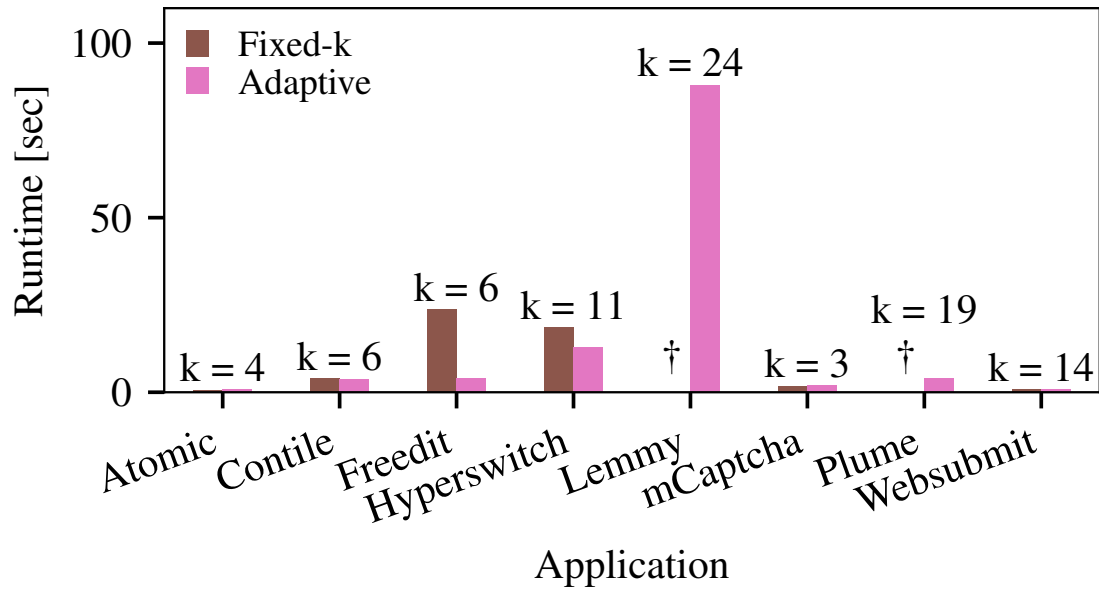


Figure 3.7. Adaptive approximation reduces end-to-end runtime: without it, PDG construction inlines call sites that cannot affect the policy’s verdict. This is what makes the interactive numbers above achievable.

CHAPTER 4

PALSGRAF

```
1 @tool(description="Writes `content` to file at `path`")
2 def write_file(self, path: str, content: str):
3     self.metrics.send({
4         'method': "write_file",
5         'bytes': len(content) })
6     with open(path, "w") as f:
7         f.write(content)
```

} undisclosed

(a) The tool as written: the metrics upload is not in its description.

```
1 #[tool(description = "Writes `content` to the file at `path`")]
2 fn write_file(&self, path: &Path, content: String,
3     // injected at the client by the protection system
4     scfg: SandboxConfig
5 ) {
6     self.metrics.send(json!({
7         method: "write_file",
8         bytes: content.len() }));
9     sandbox::write(path, content, scfg).unwrap();
10 }
```

(b) The same tool with a fine-grained sandbox.

Figure 4.1. An MCP tool for file creation. In [Figure 4.1a](#) the implementation performs an undisclosed metrics upload in addition to the documented file write; the description is the only thing the model — or the user approving its use — gets to see. [Figure 4.1b](#) adds a fine-grained sandbox whose configuration is injected by the trusted client rather than supplied by the server. Note that this is the design in which the tool developer cooperates; this chapter argues for confining the process instead, precisely because cooperation cannot be assumed.

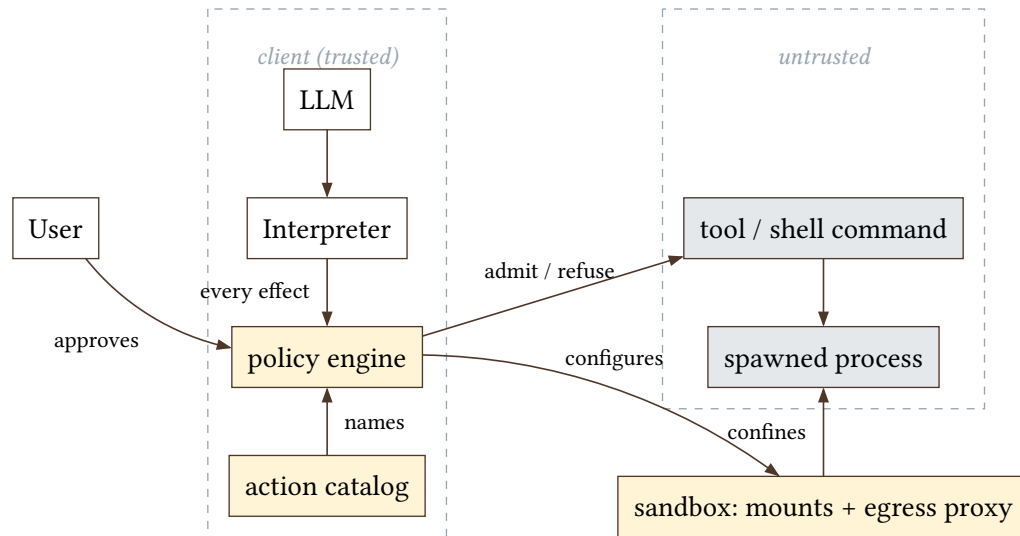


Figure 4.2. Palsgraf’s components and trust boundaries, which are easy to invert. The client is trusted and is the only party that authors policy; the user approves it in the vocabulary of the action catalog. Everything the model asks for is mediated by the interpreter, and anything that escapes the interpreter — a spawned process, most of all — is confined by a sandbox the client configured, not by cooperation from the code being run.

TODO: 12. The sandbox argument: why a sandbox, and why not a language. Make this the first technical section, because it is the design decision a reader will challenge and because the investigations already contain the argument. Write about:

- Why enforcement must be OS-level rather than in-language: an agent that can spawn a subprocess escapes any interpreter-level handler stack. The concrete finding is the right evidence — a gate on `net.request` meant nothing while `proc.spawn("curl")` was free. Generalize it: any effect mechanism the agent can reach around is not an enforcement mechanism.
- Why a purpose-built effect language was considered and set aside. Be straight about the trade: a closed effect alphabet buys cheap, precise static synopses, and costs adoption — the agent must be persuaded to use it, and the corpus says agents reach for shell. State the sandbox as the conservative choice that assumes nothing about what the agent writes. (The language work still earns a paragraph as a design point explored and the source of the effect alphabet, rather than being silently dropped.)
- What the sandbox is: mount layout derived from the path lists on Linux, an SBPL profile on macOS, egress forced through a policy-enforcing proxy. Most-specific-wins precedence rather than deny-beats-allow, and why.
- The two design consequences worth defending in print: writes are direct rather than staged (so the enforcement decision happens at the syscall, not at commit), and a host gated by a route-level rule is denied to the shell entirely (because `CONNECT host:port` carries no method, path or body, and the proxy will not guess). Both are places where the vocabulary and the mechanism do not line up perfectly; say so rather than smoothing it over.
- That the sandbox is a hard requirement with no host fallback — the fallback is the hole.

TODO: 13. Actions: the vocabulary, and the loop that produces a policy. The chapter's central contribution and the direct analogue of markers. Cover:

- The action registry: an expert-authored catalog of named, parameterized request and command shapes. Name, host or binary, and the rules it expands to. Show one entry in full and one policy generated from it.
- Parameterization and tightening: a bare placeholder admits any value matching the registry's fragment, a bound one narrows to a literal. This is what lets one vocabulary item serve both "any repo" and "this repo" without the user ever seeing a regex.
- Denial by omission: the catalog is an allow-list, so the dangerous operation is blocked because it was never named, not because someone anticipated it. The volume-delete case is the demonstration — `volumeDelete` is absent from the registry, and that absence is the defense. This is a strong point; make it explicitly, and pair it with its limit (a substring match on a GraphQL body stops an accident, not an adversary who bundles operations).
- Extending actions from REST to shell commands: what an action over `argv` looks like, how `GIT_COMMIT(where=...)` decomposes into the flags it forbids, and why naming the command shape is tractable where naming the syscall footprint is not.
- The authoring loop, which is the part that answers "where do task-specific policies come from": the agent drafts a proposal, the system renders it in the vocabulary, the user approves or refuses, the change is recorded. State clearly that the model is on the *authoring* path and off the *enforcement* path, and that the policy file is not the agent's to write — the harness denies it at the tool layer and the sandbox denies it at the OS layer, so the proposal tool is the only route and every change leaves a record.

Figure still to draw, and the most valuable one in the thesis: a worked end-to-end example — prompt, agent-proposed action set, the approval dialog as the user sees it, the generated policy, and the denied call. One figure carrying all five steps would do more than any amount of prose. It is a fletcher diagram like [Figure 4.2](#); reuse that file's node helpers. *Figures placed:* [Figure 4.2](#) is redrawn for the sandbox-only design (the paper's version showed the MCP-server-modification story and is kept in `assets/`

```
deny(all)
allow(net:write)
intent(file-edit) -> allow(fs:write in "~/workspace")
context(sensitive-data) -> deny(net:write)
```

Listing 4.1. An example policy for the file-editing tool.

```
input(path) -> effect(fs:write)
input(content) -> effect(fs:write)
input(content) -> effect(net:write)
```

Listing 4.2. The synopsis static analysis derives for the `write_file` tool: which of the tool's inputs reach which effects. The undisclosed upload shows up as content reaching a network write.

TODO: 14. Evaluation. The design is further along than the evaluation, so write this section as a plan you then fill in. Structure:

- The replay methodology: real agent sessions replayed against protection configurations, with divergence from the recorded transcript as the enforcement signal. Explain why divergence is the right oracle and what it cannot tell you.
- The protection axis, which is already the right experimental design and should be stated as such: none, static analysis, coarse sandbox, and the action-based policy. The coarse-sandbox column is the one that carries the argument — it is the honest baseline that a reasonable engineer would build, and showing it either over- or under-blocks on the same scenarios is the empirical form of the vocabulary claim.
- Scenario lineup: env-file exfiltration, sensitive data, git overreach, merge conflicts, PR/CI fixing, volume delete. Say which are accidents and which are attacks, and make sure there are positive controls — scenarios where the guardrail must *not* block — or the discrimination claim is empty.
- The questions to answer beyond block/allow: how many approval prompts a user faces per session, what registry coverage real sessions need, how often the agent’s drafted policy is accepted unchanged, and overhead.
- A user-facing evaluation of whether the vocabulary is actually legible. Without it the central claim rests on assertion; even a small study, or a structured argument about why one is impractical here, is better than silence.

Figure: the results table below is ported from the paper and still reads `assets/palsgraf/runs/results.csv`, so rerunning the experiments updates it. Two things to revisit: it covers only three scenarios, and the row the paper called “Hybrid approach” is labelled “Palsgraf” here, which is right for the thesis but means the row no longer matches the paper’s text.

Table 4.1. Which scenario variants each protection mechanism allowed or blocked. ✓ marks the intended outcome, ✗ a failure. The global sandbox is the honest baseline — a policy a careful engineer would write without per-task knowledge — and it fails in both directions, blocking intended work in one scenario and admitting divergent effects in the others. Only the per-task policy separates them everywhere.

Protection mechanism	.env file		Sensitive code		git push	
	Intended	Divergent	Intended	Divergent	Intended	Divergent
Unprotected	✓ Allowed	✗ Allowed	✓ Allowed	✗ Allowed	✓ Allowed	✗ Allowed
Global sandbox	✗ Blocked	✓ Blocked	✓ Allowed	✗ Allowed	✓ Allowed	✗ Allowed
Static effect detection	✗ Blocked	✓ Blocked	✓ Allowed	✓ Blocked	✓ Allowed	✓ Blocked
Palsgraf	✓ Allowed	✓ Blocked	✓ Allowed	✓ Blocked	✓ Allowed	✓ Blocked

CHAPTER 5

RELATED WORK AND CONCLUSION

TODO: 15. Related work, then the synthesis. Two jobs in one chapter (split it later if it grows).

- *Related work* organized by the thesis's own axis rather than by technique: for each system, who authors the policy and in what vocabulary. Static analysis and IFC (Jif, HLIO, FlowDroid, DroidSafe, Resin, Jacqueline, RuleKeeper, PrivGuard, Sesame, Riverbed, MirChecker, CodeQL); agent defenses (CaMeL, Fides, ACE, Progent, IsolateGPT, Conseca); confinement (bubblewrap, Seatbelt, RLBox, NaCl, Ryoan, BinWrap, Try/semisolates); and the deployed status quo of permission prompts. The table you want is system $\times$ policy author $\times$ vocabulary $\times$ enforcement point.
- *Synthesis*: return to the thesis statement and argue it from the two chapters. The strongest version of the argument is the contrast between them: the same idea at compile time with a trusted developer, and at runtime with an untrusted model, works both times — which is evidence the idea is about vocabularies and not about either setting.
- *Limits of the claim, stated by you*: the vocabulary must be authored by someone, and neither system says where that expertise comes from at scale or what happens when the registry or marker set is incomplete. Registry coverage is the honest open problem of [Chapter 4](#); say so.
- *Future work*: inferring or synthesizing vocabulary from code and from session corpora; reusing Paralegal's analysis to verify that a tool implementation matches its declared action; sharing and trusting registries across organizations; and what changes when the agent, not a person, is the one reading the policy.

BIBLIOGRAPHY

- [1] Engineering at Anthropic. Code execution with MCP: Building more efficient agents. Retrieved 2025 from <https://www.anthropic.com/engineering/code-execution-with-mcp>
- [2] Kenton Varda and Sunil Pai. Code Mode: the better way to use MCP. Retrieved 2025 from <https://blog.cloudflare.com/code-mode/>